

Normalização

Este capítulo tem o objetivo de compreender a verificação da qualidade de projeto de banco de dados. Inicialmente, serão discutidos alguns critérios informais para construção de um “bom” projeto de banco de dados. Um projetista de banco de dados, após adquirir algumas experiências, é capaz de organizar os atributos do banco de dados de maneira lógica e adequada.

Contudo, ainda é necessário um modo formal de análise, conhecido por normalização, que é utilizado para justificar porque um agrupamento de atributos em um esquema de relação é melhor que outro (Elmasri; Navathe, 2018). Neste texto discute-se a teoria desenvolvida com o objetivo de avaliar os esquemas relacionais para a qualidade do projeto, ou seja, deseja-se verificar formalmente porque um agrupamento de atributos em esquemas de relação é melhor que outro.

1 Critérios informais para avaliação de qualidade

O projeto de banco de dados relacional tem o propósito de gerar um conjunto de relações, denominado como esquema de banco de dados, que permite armazenar informações sem redundâncias desnecessárias e ainda permite que se recupere a informação facilmente (Silberschatz; Korth; Sudarshan, 2020). Para alcançar esses objetivos, é boa prática observar algumas diretrizes informais que são úteis como medida para garantir a qualidade do esquema da relação (Takai; Italiano; Ferreira, 2005).

- Assegurar que a semântica dos atributos esteja clara no esquema.
- Minimizar a redundância de informação.
- Reduzir a ocorrência de valores nulos.
- Evitar a geração de tuplas espúrias.

Esses critérios informais auxiliam na identificação de problemas estruturais no esquema de banco de dados relacionais.

1.1 Semântica dos atributos

Um esquema de banco de dados deve ser projetado de maneira que a semântica dos atributos seja clara e bem definida. Cada atributo deve pertencer exclusivamente à entidade ou ao relacionamento que está associado (Takai; Italiano; Ferreira, 2005). Por semântica de uma relação, entende-se que cada relação deve possuir um significado bem definido, de modo que os valores dos atributos em uma tupla sejam coerentes e representem corretamente a realidade do domínio do problema. Para ilustrar isso, observe um esquema de relação de baixa qualidade, e que deve ser evitado:

FUNCIONÁRIO(numat, nome, dtnasc, endereço, numdep, nomedep, ncheefe)

Esse esquema mistura atributos de funcionário, como numat, nome, dtnasc, endereço com os atributos de departamento, tais como numdep, nomedep e ncheefe. Isto é um esquema de relação inadequado, pois não é possível estabelecer com clareza a semântica dessa relação. Observe a tabela 1, que contém dados de funcionários.

Tabela 1 – Tabela FUNCIONÁRIO

numat	nome	dtnasc	endereço	numdep	nomedep	ncheefe
235	João da Silva	14/04/1980	Rua Margarida, 202	10	RH	Caio
121	Edna Martins	21/06/1977	Rua Magnólia, 103	20	Admin	João
345	Pedro Coutinho	29/03/1989	Rua Orquídea, 323	30	TI	Edna

Fonte: adaptado de Elmasri e Navathe (2018).



IMPORTANTE

Diretriz: cada esquema de relação deve conter apenas atributos que representam a entidade ou relacionamento que ela modela, evitando a mistura de conceitos distintos.

1.2 Minimizar a redundância de informação

A redundância de dados pode levar a inconsistências e ao aumento do espaço de armazenamento necessário. Para minimizar esse problema, a modelagem deve ser estruturada de modo que evite a repetição desnecessária de informações. Segundo Elmasri e Navathe (2018), a mistura de atributos de várias entidades pode gerar problemas conhecidos

como anomalias de atualização. São classificadas em três tipos de anomalias: anomalia de inserção, anomalia de remoção e anomalia de modificação.

- **Anomalia de inserção:** ocorre quando a inserção de novos dados em uma tabela causa inconsistência ou problemas relacionados a integridade de dados.

Observe como exemplo a tabela 1, que possui informações de funcionários e seus departamentos. Como seria possível inserir um departamento que ainda não possui funcionários? Mesmo porque o atributo “numat” da tabela de funcionário é a chave primária da relação. Isso cria a necessidade de inserir um funcionário para que uma tupla seja inserida na relação FUNCIONÁRIO.

Outro exemplo, considere a tabela 2, que é a tabela FUNC_PROJ com as informações de funcionários com informações de seus projetos.

Tabela 2 – Tabela FUNC_PROJ

numat	nomeFunc	horas	numProj	nomeProj	localProj
235	João da Silva	10	10	Projeto X	São Paulo
121	Edna Martins	35	20	Projeto Y	Itu
345	Pedro Coutinho	20	30	Projeto Z	Limeira
238	Ana Rosa Paes	15	10	Projeto X	São Paulo
242	Mário Andrade	19	20	Projeto Y	Itu
235	João da Silva	10	20	Projeto Y	Itu

Fonte: adaptado de Elmasri e Navathe (2018).

Nessa situação, a inserção de novos projetos associados a um funcionário exige a repetição dos dados desse funcionário. E a associação

de mais de um funcionário para um mesmo projeto exige a repetição dos dados do projeto.

- **Anomalia de remoção:** o que acontece se a tupla referente ao Pedro Coutinho for excluída? As informações sobre o projeto Z deixarão de existir.
- **Anomalia de modificação:** o que acontece se for necessário alterar o nome do projeto Y? Essa alteração deve ser refletida para todas as tuplas relacionadas ao projeto Y.



IMPORTANTE

Diretriz: estruturar cada esquema de relação de modo que não haja anomalia de inserção, remoção ou modificação.

1.3 Redução das ocorrências de valores nulos

A presença de muitos valores nulos pode indicar um projeto inadequado e pode afetar o desempenho das consultas. Normalmente, os atributos que possuem muitos valores nulos, podem ser separados em outra tabela. Segundo Takai, Italiano e Ferreira (2005), algumas razões justificam o valor nulo nos atributos, as quais incluem:

- Valor não aplicável ou inválido.
- Valor desconhecido, porém o valor pode existir.
- Valor indisponível no momento da inserção e que pode ser atualizado mais tarde.

Valores nulos podem desperdiçar o espaço de armazenamento e gerar dificuldades na interpretação do valor do atributo, além de causar

problemas em operações do banco de dados que envolvem combinações ou cálculos, como contagem, média, soma total, entre outros.



IMPORTANTE

Diretriz: estruturar o esquema para minimizar o uso de valores nulos sempre que possível, criando tabelas específicas quando necessário.

1.4 Evitar a geração de tuplas espúrias

Tuplas espúrias são registros inconsistentes que surgem quando esquemas de relação mal projetados precisam ser unidos (Elmasri; Navathe, 2018). Como exemplo, suponha um esquema de relação ruim, como apresenta a tabela 3.

Tabela 3 – Tabela FUNC_PROJ1

numat	nome	numProj	nomeProj	localProj
235	João da Silva	10	Projeto X	SP
235	João da Silva	20	Projeto Y	Campinas
121	Edna Martins	20	Projeto Y	Campinas
345	Pedro Coutinho	30	Projeto Z	Osasco

Fonte: adaptado de Elmasri e Navathe (2018).

Suponha que o projetista de banco de dados resolveu quebrar a FUNC_PROJ1 em duas tabelas distintas relacionadas por numat. Ele resolveu separar o atributo localProj em uma relação LOCAL_PROJ separada. Logo, as novas relações são:

`FUNC_PROJ2(numat, nome, numProj, nomeProj)` e `LOCAL_PROJ(numat, localProj)`

Observe a tabela 4 que apresenta a `FUNC_PROJ2`.

Tabela 4 – Tabela `FUNC_PROJ2`

numat	nome	numProj	nomeProj
235	João da Silva	10	Projeto X
235	João da Silva	20	Projeto Y
121	Edna Martins	20	Projeto Y
345	Pedro Coutinho	30	Projeto Z

Fonte: adaptado de Elmasri e Navathe (2018).

Agora, observe a tabela 5 que apresenta a nova tabela `LOCAL_PROJ`.

Tabela 5 – Tabela `LOCAL_PROJ`

numat	localProj
235	SP
235	Campinas
121	Campinas
345	Osasco

Fonte: adaptado de Elmasri e Navathe (2018).

Essa decisão de quebrar em duas tabelas com esses atributos não foi uma boa decisão. Caso o projetista necessite realizar um relatório envolvendo os projetos e locais de projetos, ele necessita realizar uma operação de banco de dados chamada junção, a qual irá combinar os valores das tuplas de `FUNC_PROJ2` com os valores das tuplas de `LOCAL_PROJ`, relacionados pelo atributo chave `numat`, como mostra a tabela 6.

Tabela 6 – Junção de FUNC_PROJ2 com LOCAL_PROJ

numat	nome	numProj	nomeProj	localProj
235	João da Silva	10	Projeto X	SP
235 (*)	João da Silva	20	Projeto Y	SP
235 (*)	João da Silva	10	Projeto X	Campinas
235	João da Silva	10	Projeto Y	Campinas
121	Edna Martins	20	Projeto Y	Campinas
345	Pedro Coutinho	30	Projeto Z	Osasco

Fonte: adaptado de Elmasri e Navathe (2018).

Após a junção das duas tabelas, surgiram novas tuplas que antes não existiam. Essas novas linhas, marcadas pelo (*) na tabela 6 são tuplas geradas que não refletem a realidade e são chamadas de tuplas espúrias ou tuplas adicionais. Na quebra da tabela em duas partes, a escolha dos atributos que ficariam em uma ou em outra tabela resultou na perda de informações da tabela original, ou seja, houve uma decomposição com perdas.



IMPORTANTE

Diretriz: garantir que as tabelas sejam projetadas corretamente, evitando a geração de informações incoerentes após operações de junção.

2 Dependências funcionais

Uma dependência funcional (DF) é uma restrição imposta a dois conjuntos de atributos dentro de um banco de dados. Essa restrição é definida pelo projetista, baseando-se nas regras do mundo real, e serve para garantir a consistência e qualidade dos dados.

Definição: uma dependência funcional, indicada por $X \rightarrow Y$, entre dois conjuntos de atributos de X e Y que são subconjunto de um esquema de relação R , especifica uma restrição sobre possíveis tuplas que podem formar um estado r da relação R . A restrição é: sempre que duas tuplas possuírem o mesmo valor para X , elas também terão o mesmo valor para Y , ou seja, para quaisquer duas tuplas t_1 e t_2 em r que tenham $t_1[X] = t_2[X]$, elas devem ter $t_1[Y] = t_2[Y]$ (Elmasri; Navathe, 2018). Como exemplo, suponha a tabela FUNCIONÁRIO listada na tabela 7.

Tabela 7 – tabela FUNCIONÁRIO

CPF	nome	cargo	depto
123	João da Silva	Analista	TI
456	Bruno Andrade	Gerente	Admin
123	João da Silva	Analista	TI

Fonte: elaborado pelo autor.

Nesse cenário, pode-se identificar a dependência funcional $CPF \rightarrow$ nome, cargo, depto, pois cada CPF tem um único valor para os demais atributos. Caso haja um CPF associado a dois nomes diferentes, diz-se que houve uma violação na dependência funcional.

Uma chave candidata é um conjunto mínimo de atributos que determina todos os atributos da relação. Por exemplo, dado o esquema de relação Clientes (ID, CPF, Nome, Sobrenome), as dependências funcionais são: $ID \rightarrow CPF$, nome, sobrenome e $CPF \rightarrow$ Nome, sobrenome. Aqui, os atributos ID e CPF são chaves candidatas, pois cada um deles pode determinar todos os outros atributos.

3 Normalização

Normalização é o processo utilizado para organizar os esquemas de banco de dados relacionais de forma a reduzir redundâncias (Silberschatz; Korth; Sudarshan, 2020), eliminar anomalias de inserção, atualização e exclusão, e garantir a integridade de dados (Elmasri; Navathe, 2018).

Esse processo, proposto por Codd em 1972, é realizado por etapas, chamadas de formas normais, em que um esquema de relação é analisado por meio de uma estrutura formal baseada em chaves e nas dependências funcionais entre atributos. Aplica-se uma série de testes, chamados “testes de forma normal”, cujo objetivo é certificar que esse esquema de relação atende a certas condições e regras definidas para aquela etapa da normalização. Caso o esquema de relação não seja satisfatório, ele é decomposto em esquemas de relação menores e mais simples, que atendem aos testes, ou seja, possuem propriedades desejáveis (Elmasri; Navathe, 2018).

3.1 Primeira forma normal

De acordo com Silberschatz, Korth e Sudarshan (2020), um esquema de relação R está na primeira forma normal (1FN) se os domínios de todos os atributos são atômicos. Um domínio é atômico se elementos do domínio são considerados indivisíveis. Observe a tabela 8, contendo a tabela FUNCIONÁRIO, observe que o atributo “telefones” é um valor multivalorado, ou seja, é um atributo não atômico, portanto não está na primeira forma normal. Além disso, telefones não é funcionalmente dependente da chave primária CPF.

Tabela 8 – tabela FUNCIONÁRIO não está na 1FN

CPF	nome	cargo	telefones
123	João da Silva	Analista	1234-5678, 2345-6789
456	Bruno Andrade	Gerente	3456-7890
789	Lúcio Costa	Analista	4567-8901, 5678-9012

Fonte: Elaborado pelo autor.

Segundo Elmasri e Navathe (2018), existem três técnicas para obter a primeira forma normal para a relação da tabela 8:

- Remova o atributo telefones da tabela FUNCIONÁRIO, pois viola a 1FN. Crie uma tabela TELEFONE contendo o CPF do funcionário e o número do telefone. Cada número de telefone será uma linha separada na tabela TELEFONE. A chave primária dessa tabela é {CPF, telefone}, como mostra a tabela 9. Agora as relações FUNCIONÁRIO e TELEFONE estão na primeira forma normal.

Tabela 9 – tabela TELEFONE na 1FN

CPF	telefone
123	1234-5678
123	2345-6789
456	3456-7890
789	4567-8901
789	5678-9012

Fonte: elaborado pelo autor.

- Expandir a chave de modo que haverá uma tupla separada na relação original. Nesse caso, a chave primária de FUNCIONÁRIO torna-se {CPF, telefone}. Essa solução tem desvantagem de introduzir redundância na relação, como mostra a tabela 10.

Tabela 10 – tabela FUNCIONÁRIO na 1FN

CPF	nome	cargo	telefone
123	João da Silva	Analista	1234-5678
123	João da Silva	Analista	2345-6789
456	Bruno Andrade	Gerente	3456-7890
789	Lúcio Costa	Analista	4567-8901
789	Lúcio Costa	Analista	5678-9012

Fonte: elaborado pelo autor.

- Se o número máximo de telefones for conhecido, pode-se trocar o atributo telefones por três atributos atômicos, por exemplo telefone1, telefone2 e telefone3. Essa solução normaliza a tabela FUNCIONÁRIO na primeira forma normal, porém tem a desvantagem de gerar valores nulos desnecessários, caso a maioria dos funcionários tenham menos que três números de telefones.

3.2 Segunda forma normal

Segundo Takai, Italiano e Ferreira (2005), uma relação está na segunda forma normal (2FN) se estiver na primeira forma normal (1FN) e todos os atributos não-primos dependerem totalmente da chave-primária. Por atributos não-primos, entende-se todo atributo que não é membro de chave-candidata.

Suponha um esquema de relação FUNCIONARIO_PROJETO (CPF, numProj, horasProj, nomeFunc, nomeProj, localProj). Ela está na 1FN, pois todos os atributos são atômicos. Observe as dependências de atributos não primos com relação à chave primária.

Exemplo1: {CPF, numProj} $\rightarrow$ horasProj

O atributo horasProj é atributo não-primo e depende funcionalmente da chave primária. O atributo horasProj não pode depender apenas do atributo CPF. O atributo horasProj não pode depender apenas do atributo numProj. Essa dependência funcional é total, pois qualquer remoção de um dos dois atributos CPF ou numProj invalida a dependência funcional.

Exemplo2: {CPF, numProj} $\rightarrow$ nomeFunc

O atributo nomeFunc não é totalmente dependente de {CPF, numProj}. Basta apenas o CPF $\rightarrow$ nomeFunc. Perceba que se pode remover o atributo numProj e a dependência funcional se mantém, então essa dependência é parcial. A dependência parcial viola a 2FN, pois nem todos os atributos não-primos de FUNCIONARIO_PROJETO dependem totalmente da chave primária.

Se um esquema de relação não está na 2FN, ele pode ser decomposto em esquemas de relações que estejam na segunda forma normal (2FN) através do processo de normalização. Observe as dependências funcionais.

DF1: {CPF, numProj} $\rightarrow$ horasProj

DF2: {CPF} $\rightarrow$ nomeFunc

DF3: {numProj} $\rightarrow$ nomeProj, localProj

As dependências funcionais indicam que DF1 é dependência total, enquanto DF2 e DF3 são dependências parciais. Elas podem ser utilizadas para decompor a relação FUNCIONARIO_PROJETO em 3 relações que satisfaçam a 2FN. Agora, as novas relações FUNC_PROJ1, FUNC_

PROJ2 e FUNC_PROJ3 evitam as redundâncias desnecessárias que poderiam ocorrer em FUNCIONARIO_PROJETO.

- FUNC_PROJ1 {CPE, numProj, horas}
- FUNC_PROJ2 {CPE, nomeFunc}
- FUNC_PROJ3 {numProj, nomeProj, localProj}

3.3 Terceira forma normal

Um esquema de relação está na terceira forma normal (3FN) se estiver na segunda forma normal (2FN) e todos os atributos não-primos dependerem funcionalmente apenas da chave primária. Isso significa que não deve haver dependências transitivas entre os atributos não-primos. Uma dependência funcional $X \rightarrow Z$ em um esquema de relação R é uma dependência transitiva se houver um conjunto de atributos Y que não sejam chaves candidatas nem subconjunto de chaves de R , e ocorre que $X \rightarrow Y$ e $Y \rightarrow Z$, se mantêm. Quando Y é uma chave-candidata, não existe problema com a dependência transitiva.

Caso o esquema de relação R não esteja na 3FN, ele pode ser decomposto em relações que estejam na 3FN via processo de normalização.

Suponha um esquema de relação FUNCIONARIO_PROJETO (CPE, nomeFunc, dtnasc, numProj, nomeProj, localProj). Observe as seguintes dependências funcionais.

- $CPE \rightarrow nomeFunc$
- $CPE \rightarrow dtnasc$
- $CPE \rightarrow numProj$
- $numProj \rightarrow nomeProj$
- $NumProj \rightarrow localProj$

Aplicando a normalização para 3FN tem-se:

- FUNCIONÁRIO (CPE, nomeFunc, dtnasc, numProj)
- PROJETO (numProj, nomeProj, localProj)



PARA SABER MAIS

Para maioria dos projetos de banco de dados, a aplicação da terceira forma normal é suficiente. Existem outras formas normais mais complexas, as quais incluem forma normal de Boyce Codd, a quarta forma normal e quinta forma normal. Elas estão fora do escopo deste livro. Contudo, vários autores abordam estas formas normais mais avançadas. Para saber mais leia o livro de Elmasri e Navathe (2018), Silberschatz e Korth e Sudarshan (2020).

Considerações finais

Este capítulo apresentou, inicialmente, alguns critérios informais de banco de dados, os quais são importantes para eliminar redundância, organizar melhor o espaço de armazenamento e evitar uso de valores nulos desnecessários. Para isso, o texto descreve algumas diretrizes que servem como boas práticas.

Em sequência, o texto apresentou a definição do conceito de dependências funcionais, os quais são utilizados na seção de normalização. Foi apresentada a primeira forma normal, segunda forma normal e terceira forma normal que fornecem regras visando evitar redundâncias, por meio da eliminação de anomalias e garantindo a integridade do banco de dados.

Referências

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018.

RAMAKRISHNAN, Raghu; GEHRKE, Johannes. **Sistema de gerenciamento de banco de dados**. 3. ed. Porto Alegre: AMGH, 2011.

SILBERCHATZ, Abraham; KORTH, Henry F; SUDARSHAN, S. **Sistema de Banco de Dados**. 7. ed. Rio de Janeiro: GEN LTC, 2020.

TAKAI, Osvaldo Kotaro; ITALIANO, Isabel Cristina; FERREIRA, João Eduardo. **Introdução a banco de dados**. São Paulo: Universidade de São Paulo, 2005. *E-book*. Disponível em: <https://www.ime.usp.br/~jef/apostila.pdf>. Acesso em: 25 jan. 2025.